代码块

```cpp
#include <iostream>
#include <vector>
#include <queue>

using namespace std;

typedef long long LL;

int n;
priority_queue<LL, vector<LL>, greater<LL>> heap;

int main()
{
    cin >> n;
    for(int i = 0; i < n; i++)
    {
        LL x;
        cin >> x;
        heap.push(x);
    }

    LL sum = 0;
    while(heap.size() > 1)
    {
        // 取出最小的两堆合并
        LL a = heap.top(); heap.pop();
        LL b = heap.top(); heap.pop();
        heap.push(a + b);
        sum += a + b;
    }

    cout << sum << endl;

    return 0;
}
```

8.4 区间问题

8.4.1 线段覆盖

> **题目来源：** 洛谷
>
> **题目链接：** P1803 凌乱的yyy / 线段覆盖

> **难度系数：** ★★

【题目描述】

> 现在各大 oj 上有 n 个比赛，每个比赛的开始、结束的时间点是知道的。
>
> yyy 认为，参加越多的比赛，noip 就能考的越好（假的）。
>
> 所以，他想知道他最多能参加几个比赛。
>
> 由于 yyy 是蒟蒻，如果要参加一个比赛必须善始善终，而且不能同时参加 2 个及以上的比赛。

【输入描述】

> 第一行是一个整数 n ，接下来 n 行每行是 2 个整数 $a_i, b_i\ (a_i < b_i)$ ，表示比赛开始、结束的时间。
>
> 对于 100% 的数据，$1 \le n \le 10^6, 0 \le a_i < b_i \le 10^6$

【输出描述】

> 一个整数，表示最多参加的比赛数目。

【示例一】

> 输入：
>
> ```
> 3
> 0 2
> 2 4
> 1 3
> ```
>
> 输出：
>
> ```
> 2
> ```

【解法】

按照区间左端点从小到大排序，当两个区间「重叠」的时候，我们必须要舍弃一个。为了能够「在移除某个区间后，保留更多的区间」，我们应该把「区间范围较大」的区间移除。

因此以第一个区间为基准，遍历所有的区间：

- 如果重叠，选择「最小的右端点」作为新的基准；
- 如果不重叠，那么我们就能多选一个区间，以「新区间为基准」继续向后遍历。

可以用「交换论证法」证明我们的贪心策略是最优解：

> 在从前往后扫描的过程中，当贪心解和最优解第一次出现不同决策时，关于两个区间 a, b （其中 a 在左，b 在右）的取舍，有下面两种情况：
>
> 1. a, b 两个区间不重叠：

- 贪心解会将 a 区间保留，然后以 b 区间为基准，继续向后对比别的区间；
- 最优解的选择有下面几种情况：
  a. 舍弃一个区间，那么必定不如贪心解，矛盾。
  b. 以 a 区间为基准，向后对比。那更夸张了，我们已经按照区间左端点从小到大排好序了，如果 a,b 不重叠，那么 a 与后面的所有区间都不重叠，用 a 作为基准没有一点意义。还会选出与 b 重叠的区间。

综上，如果「不重叠」的话，贪心解和最优解的决策应该是「一致」的。

2. a,b 两个区间重叠，那么无论什么解，都需要「舍弃」一个区间：

- 贪心解会保留两者「右端点较小」的区间，舍弃「右端点较大」的区间；
- 最优解的选择就是，保留「右端点较大」的区间，舍弃「右端点较小」的区间。

  如果第二种决策能在此基础上得到最优解，那么我们把「右端点较大」区间换成「右端点较小」的区间是不受影响的。

  因为「较大区间」都和后续选择的区间「没有重叠」，这个较小的区间也必定「没有重叠」。

因此，最优解可以调整成贪心解。

【参考代码】

代码块

```cpp
#include <iostream>
#include <algorithm>

using namespace std;

typedef pair<int, int> PII;

const int N = 1e6 + 10;

int n;
PII a[N];

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> a[i].first >> a[i].second;
    sort(a + 1, a + 1 + n); // 默然按照首元素比较

    int r = a[1].second, ret = 1;
    for(int i = 2; i <= n; i++)
```

```
21      {
22          int x = a[i].first, y = a[i].second;
23          if(r <= x) // 没有重叠
24          {
25              ret++;
26              r = y;
27          }
28          else // 有重叠，就选右边界最小的那一个
29          {
30              r = min(r, y);
31          }
32      }
33
34      cout << ret << endl;
35
36      return 0;
37  }
```

8.4.2 Radar Installation

题目来源： 洛谷

题目链接： Radar Installation

难度系数： ★★

【题目描述】

假设海岸线是一条无限长的直线，陆地位于海岸线的一边，大海位于海岸线的另一边。大海中有许多小岛。某安全部门为了监视这些岛上是否有敌人入侵，打算在海岸线上安装若干个雷达来检测岛屿的情况。每个雷达的覆盖范围是以雷达中心为圆心，半径为 d 的圆形区域。

我们用平面之间坐标系来表示整个区域，海岸线为 x 轴，大海位于 x 轴上方，陆地位于 x 轴下方。为了节约成本，安全部门想使用最少的雷达覆盖所有的岛屿。现在已知每个岛屿的坐标 (x, y) 和雷达的覆盖半径 d ，你的任务就是计算出能够覆盖所有岛屿的最少雷达数量。

【输入描述】

输入包含若干组数据。

每组数据的第一行有两个整数 $n(1 \leq n \leq 1000)$ 和 d ，分别表示岛屿的数量和雷达的覆盖半径，之后的 n 行，每行有两个整数，表示第 i 个岛屿的坐标 (x_i, y_i) 。

相邻的两组数据使用一个空行隔开。输入 $0\ 0$ 表示输入结束。

【输出描述】

对于每一组数据的输出格式为 $Case\ i:\ x$，表示第 i 组数据中最少需要 x 个雷达来覆盖所有的岛屿； $x = -1$ 表示这组数据无解（无法覆盖所有的岛屿）。

【示例一】

输入：

```
3 2
1 2
-3 1
2 1

1 2
0 2

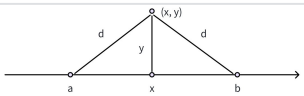
0 0
```

输出：

```
Case 1: 2
Case 2: 1
```

【解法】

如图所示，当一个岛屿的「坐标」已知，其实可以计算出：当雷达放在 x 轴的「哪段区间」时，可以覆盖到这个岛屿。



根据「勾股定理」得： ax 的长度为 $l = \sqrt{d^2 - y^2}$ ，那么雷达所处的范围就是 $[x - l, x + l]$ 。因此，针对每一个岛屿，我们都可以算出一个「能够覆盖它的区间」。

原问题就变成给定一些区间，从中选取一些区间，能够覆盖掉所有的区间，最少能够选多少个。就从「二维平面」转化成「一维线段」。

按照区间「左端点从小到大」排序，当两个区间「重叠」的时候，为了后面能够「尽可能多的选出互相重叠的区间」，我们应该把「区间范围较大」的区间移除，因为选择较大区间会造成选出来的区间「不是互相重叠」的。

因此以第一个区间为基准，遍历所有的区间：

- 如果重叠，选择「最小的右端点」作为新的基准；
- 如果不重叠，那么我们就能多选一个区间，以「新区间为基准」继续向后遍历。

可以用「反证法」证明，所有区间按照按照「左端点」排序之后，「互相重叠的区间」都是「相邻」的：

> 假设所有区间按照左端点排序之后，存在互相重叠的区间，它们是不相邻的。也就是存在 a, b, c, d 四个区间，其中 a, b, d 互相重叠，但是 a, b, c 与它们三个不是互相重叠。
>
> 设 a, b, d 区间重叠部分的范围是 $[x, y]$ ，那么 c 的位置有两种情况：
>
> - c 在 $[x, y]$ 的左侧，与实际不符：
>
> 因为如果 c 在左侧，又要与 a, b 不是互相重叠，那么 c 的右端点必须要大于 b 的左端点，那就与所有线段按照左端点排序不符；
> - c 在 $[x, y]$ 的右侧，也与实际不符：
>
> 因为如果 c 在右侧，又要与 a, b 不是互相重叠，那么 c 的左端点必须要大于 a, b 的右端点的最小值；又因为 d 与 a, b 互相重叠， d 的左端点就要小于 a, b 的右端点的最小值，与所有区间按照左端点排序不符。
>
> 综上所述，所有区间按照按照「左端点」排序之后，「互相重叠的区间」都是「相邻」的。

大家注意，虽然这道题和《线段覆盖》这道题处理区间的方式是一样的，但是「贪心的策略」是不一样的。

【参考代码】

代码块

```
#include <iostream>
#include <cmath>
```

```cpp
#include <algorithm>

using namespace std;

const int N = 1010;

int n, d;
struct node
{
    double l, r;
}a[N];

bool cmp(node& x, node& y)
{
    return x.l < y.l;
}

int main()
{
    int cnt = 0;
    while(cin >> n >> d, n && d)
    {
        cnt++;
        bool flag = false; // 有可能岛屿太远，无论如何也覆盖不到
        for(int i = 1; i <= n; i++)
        {
            double x, y; cin >> x >> y;
            if(y > d) flag = true;
            else
            {
                double t = sqrt(d * d - y * y);
                a[i].l = x - t, a[i].r = x + t;
            }
        }
        cout << "Case " << cnt << ": ";
        if(flag) cout << -1 << endl;
        else
        {
            sort(a + 1, a + 1 + n, cmp);
            int ret = 1;
            double r = a[1].r;
            for(int i = 2; i <= n; i++)
            {
                double x = a[i].l, y = a[i].r;
                if(x > r) // 没有重叠
                {
                    ret++;
```

```
50                      r = y;
51                  }
52                  else // 有重叠
53                  {
54                      r = min(r, y);
55                  }
56              }
57              cout << ret << endl;
58          }
59      }
60
61      return 0;
62  }
```

8.4.3 Sunscreen

题目来源： 洛谷

题目链接： P2887 [USACO07NOV] Sunscreen G

难度系数： ★★

【题目描述】

有 C 个奶牛去晒太阳 $(1 \leq C \leq 2500)$，每个奶牛各自能够忍受的阳光强度有一个最小值和一个最大值，太大就晒伤了，太小奶牛没感觉。

而刚开始的阳光的强度非常大，奶牛都承受不住，然后奶牛就得涂抹防晒霜，防晒霜的作用是让阳光照在身上的阳光强度固定为某个值。那么为了不让奶牛烫伤，又不会没有效果。给出了 L 种防晒霜。每种的数量和固定的阳光强度也给出来了

每个奶牛只能抹一瓶防晒霜，最后问能够享受晒太阳的奶牛有几个。

【输入描述】

第一行：两个整数： C, L；

接下来 C 行，每行两个整数，表示每个奶牛能忍受的阳光强度的最小值和最大值；

接下来 L 行，每行两个整数，表示每种防晒霜的固定阳光强度和数量

【输出描述】

输出一个整数，代表最多可以满足奶牛日光浴的奶牛数目。

【示例一】

输入：

```
3 2
```

3 10

2 5

1 5

6 2

4 1

输出：

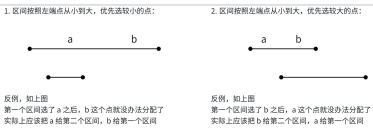
2

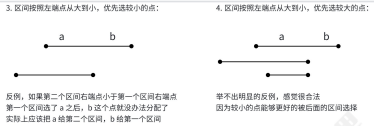
【解法】

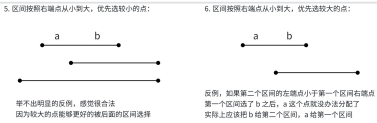
思考具体解法，从下面的情况中，筛选出没有特别明显的反例的组合：

1. 区间按照左端点从小到大 + 防晒霜从小到大（优先选小）；
2. 区间按照左端点从小到大 + 防晒霜从大到小（优先选大）；
3. 区间按照左端点从大到小 + 防晒霜从小到大（优先选小）；
4. 区间按照左端点从大到小 + 防晒霜从大到小（优先选大）；
5. 区间按照右端点从小到大 + 防晒霜从小到大（优先选小）；
6. 区间按照右端点从小到大 + 防晒霜从大到小（优先选大）。
7. 区间按照右端点从大到小 + 防晒霜从小到大（优先选小）；
8. 区间按照右端点从大到小 + 防晒霜从大到小（优先选大）。

虽然看似很多，但是很容易在错误的策略中举出「反例」。









综上所述，有两种组合没有明显的反例，分别是：

1. 区间按照「左端点从大到小」排序，防晒霜从大到小排序，「优先选择较大」的防晒霜；
2. 区间按照「右端点从小到大」排序，防晒霜从小到大排序，「优先选择较小」的防晒霜。

实际上两种情况都是正确的，我们取其一证明一下，另一种证明方式类似。

可以用「交换论证法」证明方式二的正确性：

> 从前往后依次比较「贪心解」和「最优解」针对每一个区间的决策，当找到第一个区间，它们的「分配决策不同」时：设贪心解用的是 a 防晒霜，最优解用的是 b 防晒霜，因为贪心解选的是最小的，所以有 $a \leq b$。
>
> 此时关于 a 防晒霜的使用情况，可以分以下几种情况：
>
> 1. a 防晒霜在「最优解中没有使用」，那么我们可以直接用 a 防晒霜替换 b 防晒霜，此时最优解的「最优性」并没有损失，那么贪心解就和最优解决策一致；
> 2. a 防晒霜在「最优解的后续决策中使用了」，设 b 使用的区间是 $[x_b, y_b]$，a 使用的区间是 $[x_a, y_a]$：
>
>   易得：$x_b \leq b \leq y_b$，$x_a \leq a \leq y_a$
>
>   因为我们是按照右端点从小到大排序的，所以 $y_a \geq y_b$；
>
>   综上：$x_a \leq a \leq b \leq y_b \leq y_a$。
>
>   所以 b 也可以作用于「a 作用的区间」，也就是在最优解中「a, b 可以互换」，进而就转换成贪心解。
>
> 因此，针对每一个位置，我们都可以把最优解在「不失去其最优性的前提下」，转化成「贪心解」。

【参考代码】

代码块

```cpp
#include <iostream>
#include <algorithm>

using namespace std;

const int N = 2510;

int n, m;

struct node
{
    int x, y;
}a[N], b[N];

bool cmp1(node& x, node& y)
{
    return x.y < y.y;
}

bool cmp2(node& x, node& y)
```

```
21  {
22      return x.x < y.x;
23  }
24
25  bool cmp3(node& x, node& y)
26  {
27      return x.x > y.x;
28  }
29
30  // 所有区间按照右端点从小到大排序
31  // 所有点按照从小到大排序
32  void solve1()
33  {
34      sort(a + 1, a + 1 + n, cmp1);
35      sort(b + 1, b + 1 + m, cmp2);
36
37      int ret = 0;
38      for(int i = 1; i <= n; i++)
39      {
40          int l = a[i].x, r = a[i].y;
41          for(int j = 1; j <= m; j++)
42          {
43              // 选一个最小的，符合要求的点
44              int &sp = b[j].x, &cnt = b[j].y;
45              if(!cnt || sp < l) continue;
46              if(sp > r) break;
47              cnt--;
48              ret++;
49              break;
50          }
51      }
52      cout << ret << endl;
53  }
54
55  // 所有区间按照左端点从大到小排列
56  // 所有点从大到小排列
57  void solve2()
58  {
59      sort(a + 1, a + 1 + n, cmp3);
60      sort(b + 1, b + 1 + m, cmp3);
61
62      int ret = 0;
63      for(int i = 1; i <= n; i++)
64      {
65          int l = a[i].x, r = a[i].y;
66          for(int j = 1; j <= m; j++)
67          {
```

```
            // 选一个最大的，符合要求的点
            int &sp = b[j].x, &cnt = b[j].y;
            if(!cnt || sp > r) continue;
            if(sp < l) break;
            ret++;
            cnt--;
            break;
        }
    }
    cout << ret << endl;
}

int main()
{
    cin >> n >> m;
    for(int i = 1; i <= n; i++) cin >> a[i].x >> a[i].y;
    for(int i = 1; i <= m; i++) cin >> b[i].x >> b[i].y;

    // solve1(); // 按照右端点排序
    solve2(); // 按照左端点排序
```

8.4.4 牛栏预定

题目来源： 洛谷

题目链接： P2859 [USACO06FEB] Stall Reservations S

难度系数： ★★

【题目描述】

约翰的 $N(1 \leq N \leq 50000)$ 头奶牛实在是太难伺候了，她们甚至有自己独特的产奶时段。对于某一头奶牛，她每天的产奶时段是固定的时间段 $[A, B]$（即 A 到 B，包括 A 和 B）。这使得约翰必须开发一个调控系统来决定每头奶牛应该被安排到哪个牛棚去挤奶，因为奶牛们并不希望在挤奶时被其它奶牛看见。

请帮约翰计算：如果要满足奶牛们的要求，并且每天每头奶牛都要被挤过奶，至少需要多少牛棚和每头牛应该在哪个牛棚被挤奶。如果有多种答案，输出任意一种均可。

【输入描述】

第一行为一个整数： N。

接下来 N，每行两个数字，分别代表第 i 头奶牛的产奶时段。

【输出描述】

第 1 行输出一个整数，代表需要牛棚的最少数量。

> 第 $2 \sim (N+1)$ 行，每行一个数字，第 $i+1$ 行的数字代表第 i 头奶牛将会被安排到哪个牛棚挤奶。

【示例一】

> 输入：
>
> 5
>
> 1 10
>
> 2 4
>
> 3 6
>
> 5 8
>
> 4 7
>
> 输出：
>
> 4
>
> 1
>
> 2
>
> 3
>
> 2
>
> 4

【解法】

按照「起始时间」对所有奶牛「从小到大」排序，然后「从前往后」依次安排每一头奶牛，设这头奶牛的产奶的时间区间是 $[a,b]$：

- 在已经有牛的所有牛棚里，如果「结束时间小于 a」，就可以把这头奶牛放在这个牛棚里面；如果有很多符合要求的，我们应该找一个「结束时间最大」的，这样可以将「结束时间较小」的留在后面，为后面的区间提供更多的机会。
- 如果所有已经有牛的牛棚的「结束时间都大于 a」，那么这头牛只能自己单独开一个牛棚。

这个贪心策略是比较符合我们「常识」的，尽量优的安排每一头牛。证明方法可以用「交换论证法」，前面已经叙述多次这种方式，大家可以自己尝试证明一下。我觉得重点应该是「如何实现我们这个想法」。

如何实现我们的算法：

1. 因为需要记录每一头奶牛分配的「牛棚号」，但是「排序」操作会让奶牛的「顺序」发生变化，不利于最后统计答案。因此我们可以定义一个叙述奶牛信息的「结构体」，里面存放如下信息：

- 产奶开始时间；
- 产奶结束时间；
- 排序之前的下标；
- 分配的牛棚；

这样我们对结构体排序之后，就能找到排序后的每一头奶牛所对应的「原始下标」。

2. 用什么存储每一个牛棚的「结束时间」？思考一下，我们要不断地在牛棚里面查找「小于 a 的最大值」：
   - 如果牛棚无序，我们需要遍历整个牛棚，时间复杂度吃不消；
   - 如果有序，我们可以用「二分」快速找到目标位置。

   但是用「数组」存，每次还要「排序」，效率还不如遍历，此时我们要用到 STL 提供的一个容器： set ，这个容器能够让「每次插入一个新的元素之后都是有序」的，并且「自带 $lower_bound$ 和 $upper_bound$ 」，整体的时间复杂度为 $O(logN)$ 。

   我们还需要标记每一个牛棚的「编号」，因此我们可以用 map 存储「双关键字」，一个标记结束时间，一个标记当前牛棚编号。但是有可能牛棚的结束时间一致，而 map 不能有重复元素，此时要用 $multimap$ 来存储每一个牛棚的信息。

【参考代码】

代码块

```
#include <iostream>
#include <map>
#include <algorithm>

using namespace std;

const int N = 5e4 + 10;

int n;
struct node
{
    // 存每一头牛的信息：开始时间，结束时间，排序之前的位置，分配的牛棚号
    int x, y, pos, num;
}a[N];
int cnt;
int ret[N];

multimap<int, int> st;

bool cmp(node& x, node& y)
```

```cpp
{
    return x.x < y.x;
}

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++)
    {
        cin >> a[i].x >> a[i].y;
        a[i].pos = i;
    }
    sort(a + 1, a + 1 + n, cmp);

    for(int i = 1; i <= n; i++)
    {
        int l = a[i].x, r = a[i].y;
        auto it = st.upper_bound(l);

        // 不能接在任何一个区间后面，那就新开一个区间
        if(it == st.end())
        {
            st.insert({r, ++cnt});
            a[i].num = cnt;
        }
        else // 能接在某个区间后面，那就接上去
        {
            int p = it->second;
            st.erase(it); // 把之前牛删掉
            st.insert({r, p}); // 新来的牛放进去
            a[i].num = p;
        }
    }

    cout << cnt << endl;
    for(int i = 1; i <= n; i++)
    {
        ret[a[i].pos] = a[i].num;
    }

    for(int i = 1; i <= n; i++) cout << ret[i] << endl;

    return 0;
}
```